



Raport cercetare semestrul 2

Adaptive Workplace Dynamics Simulator

Simulare exploratorie agent-based a dinamicii organizaționale

Masterand:
Pop Andrei

Îndrumător:
Anca Andreea Morar

19 iunie 2026

Cuprins

1	Descriere tehnică, obiectiv și funcționalități	2
1.1	Funcționalități implementate	2
2	Modelul de simulare	3
2.1	Variabile și ipoteze	3
2.2	Fluxul unei zile simulate	3
2.3	Turnover	4
2.4	Reproductibilitate	4
3	Implementare tehnică	5
3.1	Arhitectura motoarelor de simulare	5
3.2	Motorul Mesa	5
3.3	Comparație între implementări	5
3.4	Fluxul dashboard-ului și al exporturilor	6
3.5	Interfața aplicației	7
4	Scenarii și rezultate	8
4.1	Design experimental	8
4.2	Interpretarea metricilor	9
4.3	Configurația experimentală	9
4.4	Tabel comparativ	10
4.5	Grafic sumar	10
4.6	Interpretare	11
5	Dificultăți întâmpinate	11
6	Limitări	12
7	Direcții viitoare	12

1 Descriere tehnică, obiectiv și funcționalități

AWDS este un prototip software pentru simularea exploratorie a unei organizații formate din agenți de tip angajat. Fiecare agent are o stare internă dinamică, care include stres, burnout, productivitate, stare emoțională, energie, satisfacție profesională, echilibru muncă-viață și engagement. Simularea avansează în pași discreți, fiecare pas reprezentând o zi de lucru.

Modelul este folosit ca instrument tehnic de experimentare: utilizatorul configurează parametri, rulează scenarii, compară traiectorii și exportă datele rezultate. Leșirile sunt sintetice și trebuie citite ca rezultate interne ale modelului, nu ca date empirice despre organizații reale.

Obiectivul acestei etape este construirea unei platforme transparente, reproductibile și ușor de extins. Prototipul folosește Mesa ca motor agent-based principal și păstrează o implementare NumPy/custom pentru comparație tehnică. Accentul este pus pe fluxul complet: definirea parametrilor, execuția simulării, vizualizarea rezultatelor și exportul lor pentru raportare.

Din acest motiv, implementarea curentă urmărește patru cerințe principale:

- configurarea clară a parametrilor organizaționali, personali, sociali și legați de evenimente;
- rulări reproductibile pe baza seed-urilor explicite și a presetărilor salvate;
- export de date și grafice care pot fi folosite pentru comparații și rapoarte.

1.1 Funcționalități implementate

Funcționalitățile implementate pot fi grupate în câteva zone principale:

- configurarea numărului de agenți, zilelor simulate, seed-ului, presetărilor factory și presetărilor locale;
- controale pentru rulare, pauză/reluare, curățarea rezultatului și ajustarea intervalelor vizibile ale sliderelor;
- grafice live pentru stres, burnout, productivitate, stare emoțională, satisfacție, energie, turnover și distribuții finale;
- selector pentru motorul Mesa, motorul NumPy/custom sau rularea ambelor motoare pentru comparație;
- istoric de rulări și export în formate JSON, CSV, TXT, PNG și ZIP.

2 Modelul de simulare

Fiecare pas al simulării reprezintă o zi de lucru. La fiecare zi, modelul aplică factori organizaționali, factori personali, influență socială, evenimente aleatoare și reguli de actualizare a stării fiecărui agent.

2.1 Variabile și ipoteze

Agenții sunt sintetici și sunt inițializați cu rol, reziliență, ambiție, nevoie de suport social, sensibilitate la navetă, încărcătură familială, variabilitate a sănătății, productivitate de bază, sensibilitate la stres și rată de recuperare. Variabilele urmărite sunt:

- **stress, burnout, productivity, energy, satisfaction, work_life_balance** și **engagement**, normalizate între 0 și 1;
- **emotion**, între -1 și 1.

Modelul este intenționat simplificat: rețeaua socială rămâne statică pe durata unei rulări, evenimentele aleatoare sunt șocuri probabilistice, iar stilul de management și politica de lucru sunt modificatori de parametri. Stresul crește în funcție de cerințe și evenimente negative, dar scade prin resurse, recuperare și reziliență. Burnout-ul se acumulează mai lent, iar productivitatea poate crește la presiune moderată, dar scade la stres ridicat și burnout.

2.2 Fluxul unei zile simulate

La fiecare pas, motorul aplică aceeași ordine logică de actualizare pentru toți agenții activi. Mai întâi sunt citite setările scenariului: volum de lucru, stil managerial, autonomie, recunoaștere, conflict, flexibilitate, ședințe, navetă și probabilitatea evenimentelor. Aceste setări devin modificatori pentru cerințele și resursele zilnice ale agentului.

După calculul presiunii de bază, agentul primește influențe individuale. Reziliența și rata de recuperare reduc acumularea stresului, ambiția poate crește temporar productivitatea, iar sensibilitatea la stres și încărcătura familială amplifică efectele negative. Influența socială este apoi aplicată prin rețeaua de relații: starea emoțională și stresul vecinilor pot modifica ușor starea agentului curent.

1. se calculează presiunea zilnică produsă de scenariu;
2. se aplică factorii individuali ai agentului;
3. se propagă influența socială din rețeaua de colegi;
4. se generează evenimente aleatoare pozitive sau negative;
5. se actualizează stresul, burnout-ul, energia, satisfacția și productivitatea;
6. se verifică pragurile de turnover și, dacă este cazul, se programează înlocuirea.

Această ordine este importantă pentru reproductibilitate: aceeași configurație și același seed trebuie să producă aceeași traiectorie în interiorul acelui motor. Între Mesa și motorul NumPy/custom pot apărea diferențe deoarece activarea agenților și gestionarea rețelei nu sunt implementate identic.

2.3 Turnover

În aplicație, *turnover* înseamnă numărul cumulat de angajați simulați care părăsesc organizația. Plecarea unui agent poate apărea atunci când nivelul de burnout sau stres depășește pragurile configurate. Dacă înlocuirea este activă, un nou agent intră după o întârziere configurată și are temporar o penalizare de productivitate din cauza onboarding-ului.

2.4 Reproductibilitate

Același seed, aceeași configurație și același motor selectat duc la aceeași serie de rezultate pentru acel motor. Același seed nu produce neapărat traiectorii identice între Mesa și motorul NumPy/custom, deoarece implementarea și ordinea de activare sunt diferite.

3 Implementare tehnică

Aplicația este implementată în Python. Interfața web este realizată cu Streamlit, datele sunt organizate/exportate cu Pandas, graficele exportabile sunt generate cu Matplotlib, iar graficele interactive folosesc Plotly. Motorul principal folosește Mesa pentru model și agenți, împreună cu un spațiu de rețea bazat pe NetworkX. Implementarea NumPy/custom oferă o a doua rulare a acelorași reguli pentru comparație.

3.1 Arhitectura motoarelor de simulare

Prototipul separă logica de simulare în spatele unei interfețe de motor. Mesa este implementarea principală a modelului agent-based, iar dashboard-ul o accesează prin aceeași interfață folosită de orice motor disponibil: avansarea cu o zi, rularea până la final, întoarcerea unui snapshot live și întoarcerea rezultatului final. `engines.py` mapează selecția din interfață la implementarea care trebuie rulată.

Pe lângă motorul Mesa, a fost scris și un motor NumPy, numit *custom* în interfață și implementat în `engine.py`. Acesta gestionează direct lista de agenți, rețeaua socială, evenimentele aleatoare, actualizările zilnice, turnover-ul, înlocuirile și metricile agregate. Rolul lui este de motor de comparație: permite verificarea tendințelor produse de Mesa într-o implementare mai simplă și mai ușor de inspectat.

3.2 Motorul Mesa

Motorul Mesa este implementat în `mesa_engine.py`. În această versiune, mediul de lucru este reprezentat ca model Mesa, angajații sunt agenți Mesa, iar relațiile sunt stocate într-un network grid bazat pe NetworkX. Această structură păstrează modelul aproape de paradigma agent-based folosită de bibliotecă și evită adaptarea forțată a Mesa la o arhitectură custom.

Pentru comparație, motorul NumPy/custom întoarce aceeași schemă de ieșire ca Mesa: configurație, serie temporală agregată, stări finale, jurnal de evenimente și sumar final. Scopul nu este obținerea unor rezultate identice, ci verificarea faptului că tendințele principale apar și într-o implementare separată.

3.3 Comparație între implementări

Cele două motoare sunt tratate ca implementări separate ale aceleiași specificații de model. Mesa oferă structura agent-based principală, iar motorul NumPy/custom funcționează ca referință transparentă pentru verificarea rezultatelor. Tabelul următor rezumă diferențele tehnice relevante.

Aspect	Motor Mesa	Motor NumPy/custom
Rol în proiect	Implementarea principală agent-based, bazată pe model și agenți Mesa.	Implementare de comparație, scrisă explicit pentru inspectarea regulilor.
Reprezentarea agenților	Agenți Mesa conectați într-un spațiu de rețea NetworkX.	Obiecte gestionate direct într-o listă și actualizate de motor.
Activare	Pașii zilnici sunt coordonați prin modelul Mesa.	Motorul controlează direct ordinea de actualizare și agregarea.
Ieșire	Produce seria temporală, stările finale, evenimentele și sumarul final.	Produce aceeași schemă de ieșire pentru comparații și exporturi.
Utilitate	Păstrează modelul aproape de paradigma bibliotecii Mesa.	Face ecuațiile și transformările mai ușor de urmărit în cod.

Tabela 1: Comparație tehnică între motorul Mesa și motorul NumPy/custom.

Separarea aceasta ajută și la mentenanță. Dashboard-ul nu trebuie să știe cum sunt activați agenții în fiecare motor, ci doar să primească aceeași structură de date la final. În consecință, graficele, exporturile, istoricul rulărilor și tabelul comparativ pot fi reutilizate fără ramuri speciale pentru fiecare implementare.

3.4 Fluxul dashboard-ului și al exporturilor

Dashboard-ul Streamlit susține fluxul de lucru exploratoriu: alegerea unei presetări, ajustarea parametrilor, rularea simulării, compararea scenariilor și exportul rezultatelor. După finalizarea unei rulări, interfața afișează metrici sumar, grafice de tip serie temporală și distribuții finale.

3.5 Interfața aplicației

Interfața aplicației este organizată astfel încât configurarea, rularea și interpretarea rezultatelor să fie disponibile în același flux de lucru. Sidebar-ul conține presetările, controalele de rulare și parametrii principali ai modelului, iar zona centrală afișează tab-urile pentru rulare, comparare, export și descrierea modelului.

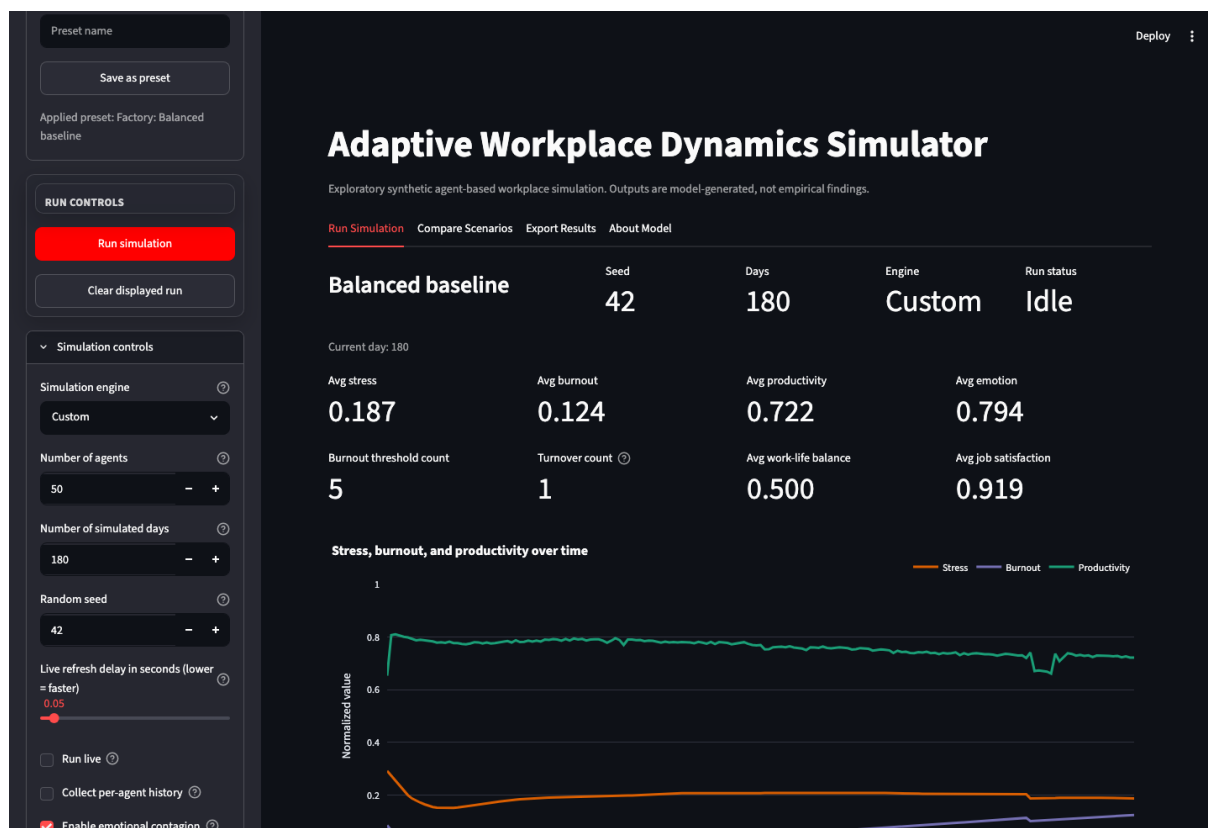


Figura 1: Tab-ul *Run Simulation* după rularea scenariului baseline.

Tab-ul de comparare rulează aceleași presetări cu aceeași configurație globală și afișează rezultatele într-un tabel agregat. Această vedere este utilă pentru inspectarea rapidă a diferențelor dintre scenarii înainte de exportul datelor.

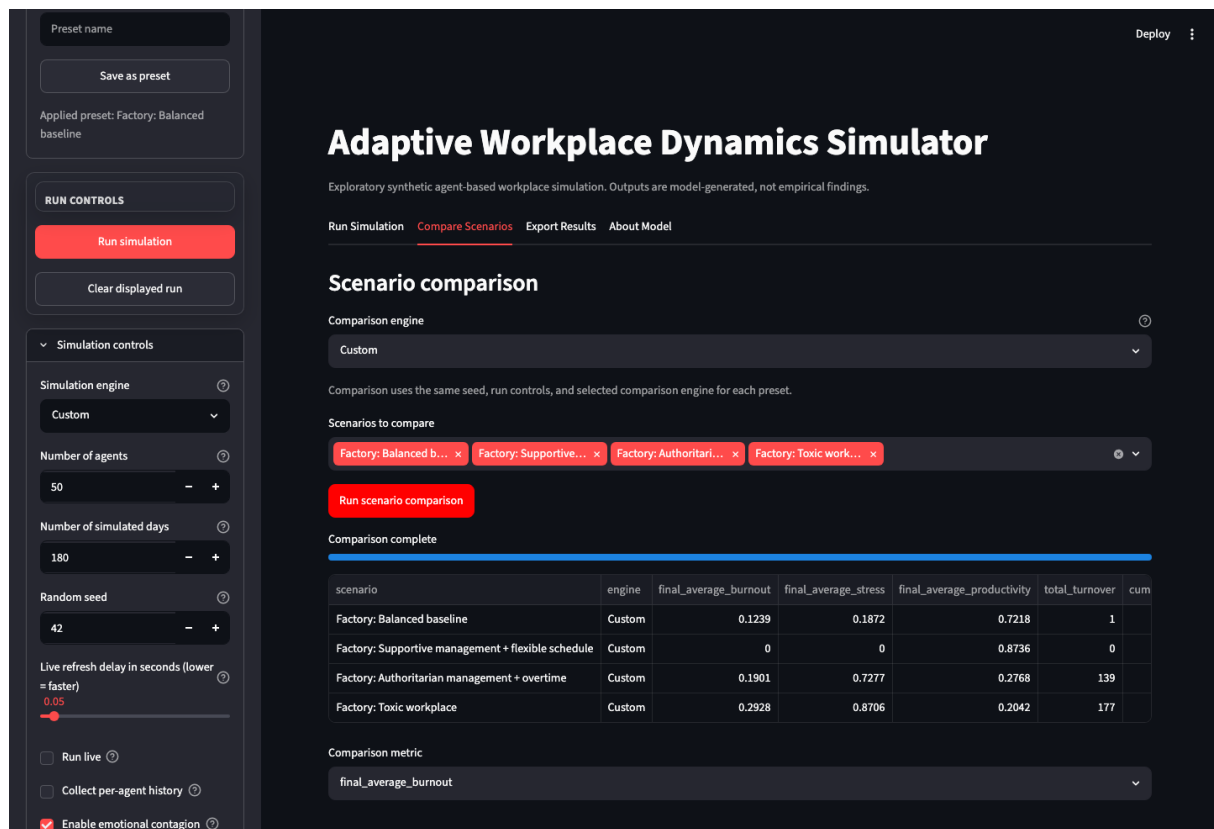


Figura 2: Tab-ul *Compare Scenarios* după finalizarea unei comparații de scenarii.

4 Scenarii și rezultate

Această secțiune prezintă o rulare comparativă a presetărilor principale din prototip. Scopul ei este să demonstreze fluxul tehnic de analiză, nu să stabilească o concluzie empirică despre mediile de lucru reale.

Fluxul de comparație este central pentru prototip. Scenariile pot fi rulate cu motorul Mesa, cu motorul NumPy/custom sau cu ambele motoare. Astfel se obține o imagine mai clară asupra întrebării de cercetare: dacă efectul observat ține de scenariu sau de diferențele introduse de motorul de simulare.

4.1 Design experimental

Au fost selectate patru presetări: baseline, suportiv + flexibil, autoritar + overtime și toxic. Ele acoperă un caz neutru, unul favorabil și două cazuri de presiune ridicată. Pentru fiecare rulare s-au păstrat constante numărul de agenți, numărul de zile, seed-ul și controalele generale, schimbând doar presetarea și motorul de simulare.

Metricile urmărite sunt stresul final, burnout-ul final, productivitatea finală, turnover-ul și burnout-ul cumulativ. Ele trebuie citite ca indicatori interni ai modelului, utili pentru compararea scenariilor sintetice, nu ca valori empirice despre organizații reale.

4.2 Interpretarea metricilor

Metricile exportate rezumă stări diferite ale organizației simulate. Ele sunt folosite împreună, deoarece o singură valoare poate ascunde dinamici importante. De exemplu, două scenarii pot avea burnout final apropiat, dar unul poate fi mult mai instabil pe parcurs.

Metrică	Rol în interpretare
Stres final	Nivelul mediu de presiune la finalul rulării.
Burnout final	Efectul acumulat al stresului susținut.
Productivitate finală	Rezultatul operațional al agenților activi la final.
Turnover total	Numărul cumulativ de plecări produse de pragurile modelului.
Burnout cumulativ	Aria aproximativă a burnout-ului în timp, utilă pentru scenarii cu traiectorii diferite.

Tabela 2: Metricile principale folosite în comparația scenariilor.

4.3 Configurația experimentală

- Număr agenți: 50
- Număr zile simulate: 180
- Seed folosit: 20260618
- Motoare comparate: Custom și Mesa
- Scenarii comparate: Balanced baseline, Supportive management + flexible schedule, Authoritarian management + overtime și Toxic workplace
- Parametri modificați față de presetări: niciunul; s-au păstrat constante controalele globale ale rulării.

Datele brute pentru această secțiune au fost exportate ca fișiere CSV în directorul data/ al raportului. Turnover-ul este raportat ca număr cumulativ de plecări pe durata rulării. Deoarece înlocuirea angajaților este activă, valoarea poate depăși numărul inițial de agenți în scenariile severe.

4.4 Tabel comparativ

Scenariu	Motor	Stres final	Burnout final	Productivitate finală	Turnover	Burnout cumulativ
Baseline	Custom	0.144	0.078	0.758	4	8.56
Baseline	Mesa	0.182	0.096	0.744	2	7.57
Suportiv + flexibil	Custom	0.000	0.000	0.872	0	0.20
Suportiv + flexibil	Mesa	0.000	0.000	0.863	0	0.21
Autoritar + overtime	Custom	0.780	0.257	0.228	132	41.69
Autoritar + overtime	Mesa	0.829	0.202	0.244	121	45.40
Toxic	Custom	0.894	0.302	0.181	188	50.05
Toxic	Mesa	0.877	0.258	0.185	185	54.95

Tabela 3: Rezultate sintetice comparative obținute din simulările AWDS pentru 50 de agenți, 180 de zile și seed-ul 20260618.

4.5 Grafic sumar

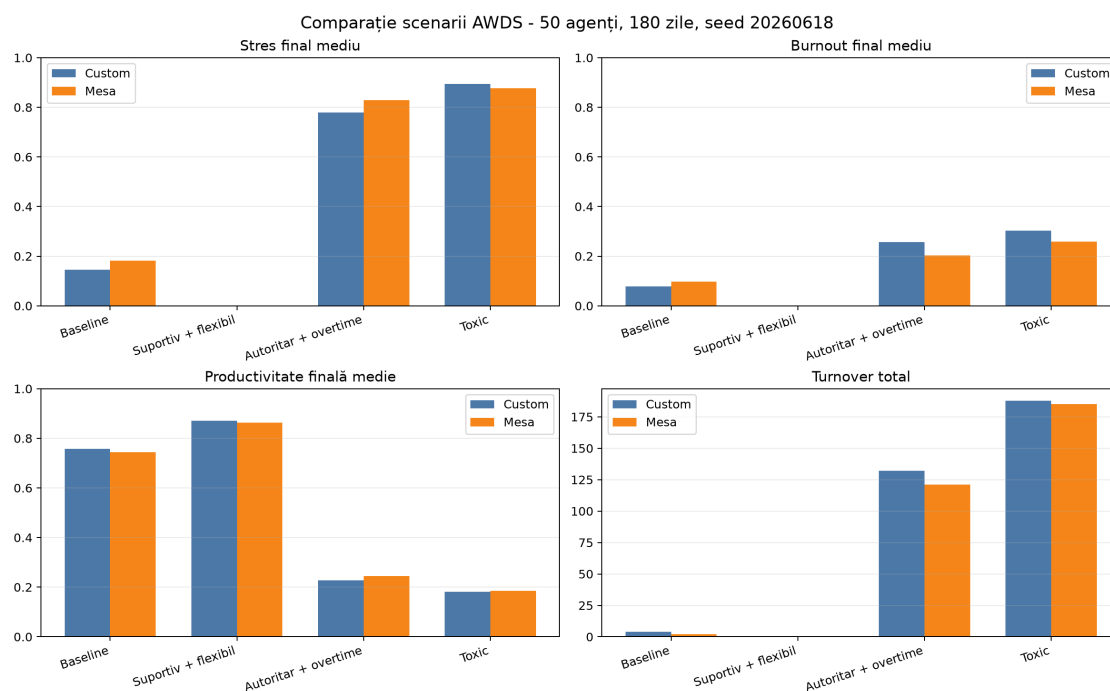


Figura 3: Comparație finală între scenariile AWDS și cele două motoare de simulare.

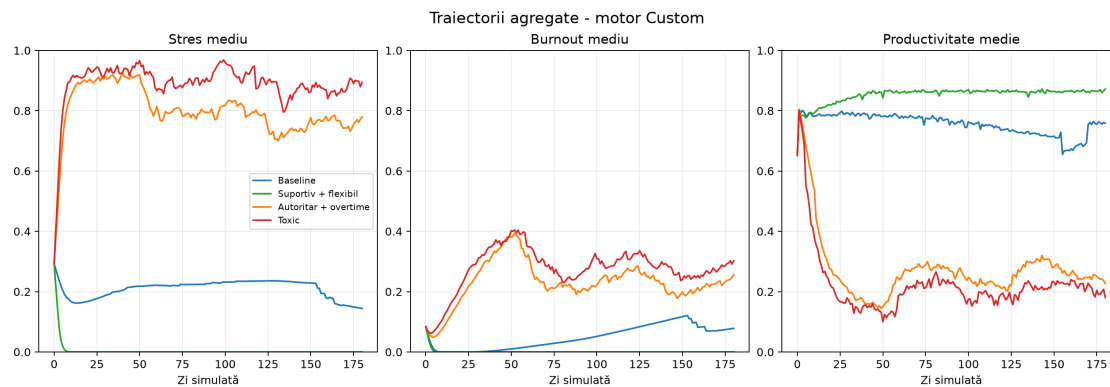


Figura 4: Traectorii agregate pentru scenariile rulate cu motorul custom.

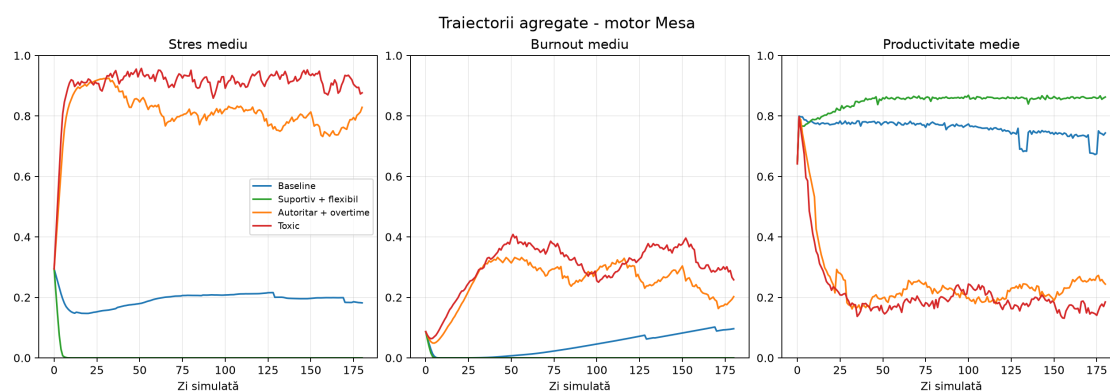


Figura 5: Traectorii agregate pentru scenariile rulate cu motorul Mesa.

4.6 Interpretare

Rezultatele separă clar scenariul suportiv de cele cu presiune ridicată. *Supportive management + flexible schedule* menține stresul și burnout-ul aproape de zero și păstrează productivitatea ridicată. În schimb, *Authoritarian management + overtime* și *Toxic workplace* cresc stresul, reduc productivitatea și generează turnover cumulat mare.

Turnover-ul poate depăși numărul inițial de agenți deoarece înlocuirea este activă: un angajat plecat este înlocuit, iar înlocuitorul poate pleca la rândul lui. Compararea motoarelor arată aceeași direcție generală a rezultatelor, chiar dacă valorile nu sunt identice, ceea ce susține fluxul tehnic de comparație fără a valida empiric modelul.

5 Dificultăți întâmpinate

Principala dificultate a fost proiectarea unui model suficient de expresiv încât să nu reducă dinamica organizațională la reguli triviale. Modelul trebuia să permită situații nuanțate: presiunea moderată poate crește productivitatea pe termen scurt, în timp ce presiunea ridicată și susținută poate produce burnout și scădere de performanță.

O altă dificultate a fost balansarea parametrilor astfel încât scenariile să producă diferențe vizibile fără să ajungă imediat în extreme. A fost necesară separarea între stres, burnout, energie, satisfacție și productivitate, deoarece aceste variabile nu se mișcă toate cu aceeași viteză și nu răspund identic la aceiași factori.

La nivel tehnic, interfața, exporturile și motorul NumPy/custom au trebuit aliniate la schema de ieșire folosită de Mesa. Astfel, graficele, istoricul rulărilor și tabelele comparative pot folosi același cod indiferent de motorul selectat.

6 Limitări

Modelul folosește date sintetice și nu este calibrat pe date empirice reale. Relațiile sociale sunt reprezentate printr-o rețea statică, evenimentele sunt probabilistice, iar trăsăturile agenților sunt generate din distribuții simple. Rezultatele pot descrie comportamentul intern al prototipului, dar nu pot fi tratate ca dovezi despre companii reale.

7 Direcții viitoare

Direcția cea mai importantă este calibrarea modelului pe date reale, de exemplu printr-un formular despre stres perceput, volum de lucru, suport managerial, autonomie, echilibru muncă–viață, satisfacție și intenția de a părăsi organizația. Aceste răspunsuri ar permite ajustarea parametrilor și compararea tendințelor sintetice cu date anonimizate.

- calibrarea parametrilor pe baza răspunsurilor colectate;
- analiza de sensibilitate pentru a vedea ce parametri influențează cel mai mult rezultatele;
- extinderea modelului cu echipe, departamente și roluri organizaționale;
- salvarea persistentă a rulărilor și generarea automată de rapoarte.